

Universidad Peruana de Ciencias Aplicadas



INFORME DE TRABAJO FINAL

Data Visualization

Carrera:

Ciencias de la Computación

Sección:

18519

Alumnos:

(U20221C621) Gomez Rubina, Luis David

(U20191C439) Miranda Cardenas, Sofia Gabriel

(U201616851) Olivera Alvarez, Lizbeth Teresita

2026

Table of Contents

Introduction.....	3
Problem Statement.....	3
Product Question.....	3
Source Inventory.....	3
Schema Draft.....	3
Entity Tables.....	3
Keys (Primary and Foreign Keys).....	4
Relationships and Joins.....	4
ERD.....	5
Data Dictionary.....	5
Raw Dataset Overview.....	5
Tables Description.....	6
Preprocessing.....	8
Data Loading and Initial Inspection.....	8
Duplicate Handling.....	8
Variable Identification.....	9
Missing Values Check.....	9
Date Column Conversion.....	9
Scale Analysis.....	9
Ethics and Access Note.....	9

Introduction

This project focuses on building a data intelligence pipeline for the healthcare sector. Using a synthetic dataset of 55,500 patient records, we aim to implement a complete lifecycle from automated ingestion to advanced entity clustering and graph-based medical analysis.

Problem Statement

The core challenge is the lack of structured, high-integrity data in hospital management. This project addresses the need for a "Catalog Layer" that cleans raw data, standardizes medical categories, and handles high-cardinality features, providing a foundation for predictive clinical insights.

Product Question

Is it possible to cluster patient profiles and treatment pathways to identify patterns between medical conditions, medication types, and hospital admission outcomes?

Source Inventory

- **Primary Source:** healthcare_dataset.csv (Synthetic Healthcare Data).
- **Entity Count:** Patients (unique individuals), Doctors, and Healthcare Providers.
- **Key Features:** Medical Condition, Admission Type, Medication, and Test Results.
- **Temporal Scope:** Continuous tracking from Admission Date to Discharge Date.

Dataset url	https://www.kaggle.com/datasets/prasad22/healthcare-dataset
-------------	---

Schema Draft

Entity Tables

The data model is composed of multiple entity tables derived from the original dataset. Each table represents a key component of the healthcare domain and is designed to support relational and analytical operations.

The main entities include:

- **patients:** stores demographic information about each patient, such as age, gender, and blood type.
- **hospitals:** contains identifiers and names of healthcare institutions.
- **doctors:** includes information about attending medical professionals.
- **medical_conditions:** represents diagnosed health conditions.
- **medications:** represents prescribed treatments.
- **medical_records:** serves as the central table that captures each clinical event, including patient interactions, diagnosis, treatment, and hospital-related data.

Keys (Primary and Foreign Keys)

The schema uses primary keys (PK) to uniquely identify each record within a table, and foreign keys (FK) to establish relationships between tables.

Primary Keys (PK):

- patients.patient_id
- hospitals.hospital_id
- doctors.doctor_id
- medical_conditions.condition_id
- medications.medication_id
- medical_records.record_id

Foreign Keys (FK):

- medical_records.patient_id → patients.patient_id
- medical_records.hospital_id → hospitals.hospital_id
- medical_records.doctor_id → doctors.doctor_id
- medical_records.condition_id → medical_conditions.condition_id
- medical_records.medication_id → medications.medication_id

Relationships and Joins

The schema defines one-to-many relationships between entity tables and the central **medical_records** table.

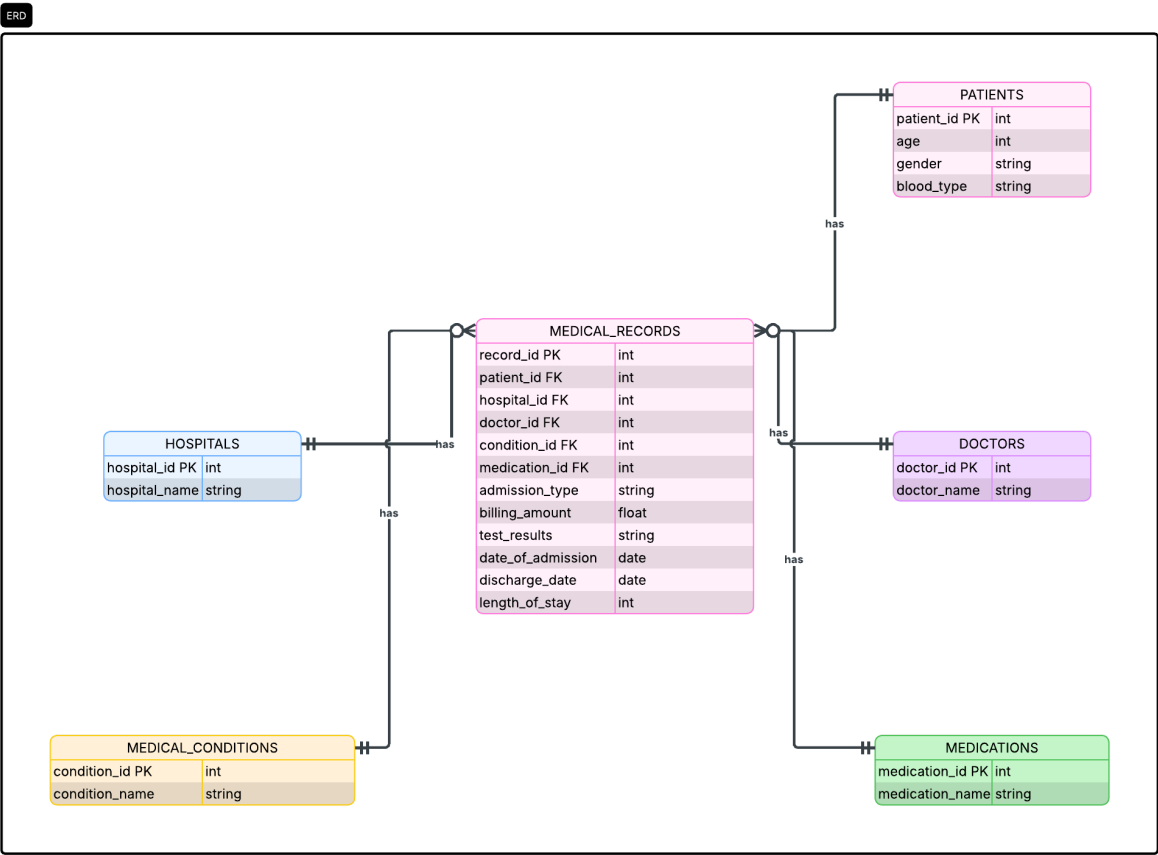
- One patient can have multiple medical records.
- One hospital can have multiple medical records.
- One doctor can be associated with multiple medical records.
- One medical condition can appear in multiple records.
- One medication can appear in multiple records.

These relationships are implemented through foreign keys and enable efficient joins for analytical purposes.

Example joins include:

- patients ↔ medical_records (via patient_id)
- hospitals ↔ medical_records (via hospital_id)
- medical_conditions ↔ medical_records (via condition_id)
- medications ↔ medical_records (via medication_id)

ERD



The schema transforms the original dataset into a normalized relational model centered on medical_records. Key entities such as patients, hospitals, doctors, conditions, and medications are separated into individual tables and connected through foreign keys, enabling scalable analysis, clustering, and graph-based modeling.

Data Dictionary

Raw Dataset Overview

Column Name	Description	Data Type
Name	Patient's name	string
Age	Age of the patient	integer
Gender	Gender of the patient (Male, Female)	categorical
Blood Type	Blood type of the patient	categorical

Medical Condition	Diagnosed medical condition	categorical
Date of Admission	Date of patient admission	datetime
Doctor	Attending doctor's name	string
Hospital	Hospital name	string
Insurance Provider	Insurance company	categorical
Billing Amount	Total billing amount	float
Room Number	Room assigned to the patient	integer
Admission Type	Type of admission (Emergency, Elective, Urgent)	categorical
Discharge Date	Date of patient discharge	datetime
Medication	Prescribed medication	categorical
Test Results	Results of medical tests	categorical

The raw dataset was transformed into multiple normalized tables to support relational analysis, clustering, and graph-based modeling.

Tables Description

Table: Patients

The patients table contains demographic information for each individual in the dataset. It includes attributes such as age, gender, and blood type, and each patient is uniquely identified by a `patient_id`. This table serves as a core entity for analyzing patient-level patterns.

Column	Type	Description
<code>patient_id</code>	int	unique identifier for each patient

age	int	age of the patient
gender	categorical	patient gender
blood_type	categorical	blood type of the patient

Table: Hospitals

The hospitals table stores information about healthcare institutions where patients receive treatment. Each hospital is uniquely identified by a `hospital_id` and is linked to medical records through relational keys.

Column	Type	Description
hospital_id	int	unique identifier for each hospital
hospital_name	string	name of the hospital

Table: Doctors

The doctors table contains information about medical professionals responsible for patient care. Each doctor is uniquely identified by a `doctor_id` and may be associated with multiple medical records.

Column	Type	Description
doctor_id	int	unique identifier for each doctor
doctor_name	string	name of the doctor

Table: medical_conditions

The `medical_conditions` table represents the set of diagnosed health conditions. Each condition is assigned a unique `condition_id`, enabling consistent referencing and reducing redundancy across the dataset.

Column	Type	Description
condition_id	int	unique identifier for the condition

condition_name	categorical	name of the medical condition
-----------------------	-------------	-------------------------------

Table: Medications

The medications table includes the different treatments or drugs prescribed to patients. Each medication is uniquely identified by a medication_id and is linked to medical records for analysis of treatment patterns.

Column	Type	Description
medication_id	int	unique identifier for the medication
medication_name	categorical	name of the medication

Table: medical_records

The medical_records table is the central fact table of the dataset. It captures each clinical event, including patient interactions with hospitals and doctors, diagnoses, treatments, admission details, and billing information. This table connects all other entities through foreign keys and enables comprehensive analytical tasks such as clustering, pattern discovery, and graph-based modeling.

Column	Type	Description
record_id	int	unique identifier for each record
patient_id	int	reference to patients table
hospital_id	int	reference to hospitals table
doctor_id	int	reference to doctors table
condition_id	int	reference to medical_conditions table
medication_id	int	reference to medications table
admission_type	categorical	type of admission (Emergency, Elective, Urgent)
billing_amount	float	total cost of treatment
test_results	categorical	results of medical tests

date_of_admission	datetime	admission date
discharge_date	datetime	discharge date
length_of_stay	int	number of days hospitalized

Preprocessing

Data Loading and Initial Inspection

The dataset, healthcare_dataset.csv, was loaded into a Pandas DataFrame. An initial inspection using df.info() revealed 15 columns with various data types, primarily object (strings), int, and float.

```
1 df.info()

*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 55500 entries, 0 to 55499
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Name                   55500 non-null  object
1   Age                   55500 non-null  int64
2   Gender                55500 non-null  object
3   Blood Type            55500 non-null  object
4   Medical Condition     55500 non-null  object
5   Date of Admission     55500 non-null  object
6   Doctor                55500 non-null  object
7   Hospital              55500 non-null  object
8   Insurance Provider    55500 non-null  object
9   Billing Amount         55500 non-null  float64
10  Room Number           55500 non-null  int64
11  Admission Type        55500 non-null  object
12  Discharge Date        55500 non-null  object
13  Medication            55500 non-null  object
14  Test Results          55500 non-null  object
dtypes: float64(1), int64(2), object(12)
memory usage: 6.4+ MB
```

Duplicate Handling

A check for duplicate rows identified 534 duplicate entries. These duplicates were subsequently removed using df.drop_duplicates(inplace=True), resulting in a cleaned dataset of 54,966 unique rows.

```
1 df.drop_duplicates(inplace=True)
2 print(f"Number of rows after removing duplicates: {len(df)}")

Number of rows after removing duplicates: 54966
```

Variable Identification

Columns were categorized into numerical and categorical types:

- Categorical Columns: Name, Gender, Blood Type, Medical Condition, Date of Admission, Doctor, Hospital, Insurance Provider, Admission Type, Discharge Date, Medication, Test Results.
- Numerical Columns: Age, Billing Amount, Room Number.

```
1 categorical_cols = df.select_dtypes(include='object').columns
2 numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
3
4 print("Categorical Columns:", categorical_cols.tolist())
5 print("Numerical Columns:", numerical_cols.tolist())
```

Categorical Columns: ['Name', 'Gender', 'Blood Type', 'Medical Condition', 'Date of Admission', 'Doctor', 'Hospital', 'Insurance Provider', 'Admission Type', 'Discharge Date', 'Medication', 'Test Results']
Numerical Columns: ['Age', 'Billing Amount', 'Room Number']

Missing Values Check

No missing values were found across any of the columns in the dataset, eliminating the need for imputation.

```
1 print('Missing values per column:')
2 print(df.isnull().sum())
```

```
... Missing values per column:
Name      0
Age       0
Gender    0
Blood Type 0
Medical Condition 0
Date of Admission 0
Doctor    0
Hospital  0
Insurance Provider 0
Billing Amount 0
Room Number 0
Admission Type 0
Discharge Date 0
Medication 0
Test Results 0
dtype: int64
```

Date Column Conversion

The columns Date of Admission and Discharge Date were converted from object (string) to datetime data type using `pd.to_datetime`. This is crucial for any time-based analysis, such as calculating the length of stay.

```

1 df['Date of Admission'] = pd.to_datetime(df['Date of Admission'])
2 df['Discharge Date'] = pd.to_datetime(df['Discharge Date'])
3
4 print('Data types after date conversion:')
5 display(df[['Date of Admission', 'Discharge Date']].info())

```

```

Data types after date conversion:
<class 'pandas.core.frame.DataFrame'>
Index: 54966 entries, 0 to 55499
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Date of Admission  54966 non-null  datetime64[ns]
1   Discharge Date    54966 non-null  datetime64[ns]
dtypes: datetime64[ns](2)
memory usage: 1.3 MB
None

```

Scale Analysis

- Number of Rows and Columns: 55,500 initial records and 15 columns. After removing duplicates, the dataset maintains 54,966 clean records.
- Missingness: The dataset is complete with 0% missing values, ensuring high reliability for statistical modeling.
- Sparsity or Memory Estimation: This is a dense dataset with a memory footprint of approximately 6.4 MB, allowing for high-performance processing in standard environments.

Ethics and Access Note

- Data Origin: The data is a synthetic dataset designed for educational and data science research.
- Usage Justification: We are utilizing this dataset under academic license for the "Domain Discovery, Recommendation, and Graph Intelligence" course.
- Personal Data Risks and Mitigation: While the data contains "Name" fields, they are synthetically generated and do not correspond to real individuals (PII). To prevent any accidental correlation, these identifiers will be treated as non-functional tokens or excluded from the final feature vectors.